Thadomal Shahani Engineering College, Bandra

Name: Bhoomi Ramrakhiani Subject: ASD&D

T.E. Batch: T23 Sem: 5th Roll No: 2301095

Date: 4th August 2026

Department of Artificial Intelligence & Data Science

Experiment 2: Containerisation using Docker

Aim: To containerise the application using Docker.

Learning Outcome (LO4):
Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Docker is an open-source containerization platform that enables applications to run in isolated environments called containers. A container includes the application along with all its required libraries, dependencies, and configuration files, ensuring consistent execution across different systems. Since containers share the host operating system's kernel, they are lightweight, portable, and start much faster than traditional virtual machines.

A Docker image is a read-only template that contains everything needed to create a container, including the operating system, software packages, and application files. Images are stored in repositories such as Docker Hub and can be downloaded using the docker pull command. Once an image is available locally, it can be used to create one or more containers.

Docker provides several commands for managing images and containers. The docker --version command is used to verify the installed Docker version, while docker images displays all locally available Docker images. The docker search command allows users to search Docker Hub for images, and docker pull downloads the required image.

Containers are created and started using the docker run command. Running a container interactively (docker run -it ubuntu) opens a Linux shell inside the container. Users can execute commands such as pwd, ls, mkdir, cd, touch, echo, and cat to perform file operations.

Containerization is a technology that packages an application along with all its dependencies into a single unit, ensuring consistent performance across different environments. It is lightweight, fast, and efficient compared to virtual machines.

Implementation:

Step 0: Open Docker Desktop

Open Start Menu

Search Docker Desktop

Open it and wait until it shows Running

Output:
(Insert Screenshot Here)

Step 1: Open Terminal

Press Windows + S

Type cmd or PowerShell

Open it

Step 2: Check Docker Version

Command: docker --version

Output:

```
C:\Users\15L>docker --version
Docker version 25.0.3, build 4debf41
```

Step 3: List Docker Images

Command: docker images

Output:

```
C:\Users\15L>docker images
REPOSITORY      TAG       IMAGE ID       CREATED         SIZE
my-flask-app    latest    08c33121e825   28 hours ago    133MB
my_custom_app   latest    08c33121e825   28 hours ago    133MB
ubuntu          latest    de7345b16e94   8 days ago      100MB
alpine          latest    d529dd0c6e55   5 weeks ago     8.42MB
mysql           8.0       6cd09145362d   2 months ago    799MB
nginx           latest    fd204fe2f750   4 months ago    161MB
hello-world     latest    1b44b5a3e06a   11 months ago   10.1kB
ubuntu          <none>    ca2b0f26964c   2 years ago     77.9MB
```

Step 4: Search Ubuntu Image

Command: docker search ubuntu

Output:

```
C:\Users\15L>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

Step 5: Pull Ubuntu Image

Command: docker pull ubuntu

Output:

```
C:\Users\15L>docker run -it ubuntu
root@16413dce4ead:/#
```

Step 6: Run Container

Command: docker run -it

ubuntu

Step 7: Execute Linux Commands
Commands:

pwd ls mkdir myfolder cd myfolder touch
file.txt echo "Hello Docker" > file.txt cat
file.txt Output:

```
root@5f13e7013492:/# pwd
/
root@5f13e7013492:/# mkdir myfolder
root@5f13e7013492:/# cd myfolder
root@5f13e7013492:/myfolder# touch file.txt
root@5f13e7013492:/myfolder# echo "Hello Docker" > file.txt
root@5f13e7013492:/myfolder# cat file.txt
Hello Docker
root@5f13e7013492:/myfolder# exit
exit
```

Step 8: Exit Container

Command: exit

Output:

```
root@16413dce4ead:/# exit
exit
```

Step 9: Check Running Containers

Command: docker ps Output:

```
C:\Users\L15>docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS          PORTS     NAMES
2b468968ffb7   ubuntu    "/bin/bash"   17 minutes ago   Up 17 minutes             modest_hellman
```

Step 10: Check All Containers

Command: docker ps -a Output:

```
C:\Users\L15>docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                      PORTS     NAMES
5f13e7013492   ubuntu    "/bin/bash"   5 minutes ago    Exited (0) 17 seconds ago             quizzical_mahavira
2b468968ffb7   ubuntu    "/bin/bash"   17 minutes ago   Up 17 minutes                         modest_hellman
C:\Users\L15>
```

Conclusion:

In this experiment, we learned the fundamentals of Docker by performing basic image and container operations. We verified the Docker installation, searched and downloaded Docker images, and created a container using the Ubuntu image. We also executed basic Linux commands inside the container. This experiment helped us understand Docker images, containers, and containerization, and how Docker provides an efficient and isolated environment for running applications.